



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIML for Robotics

Module 3

Localization

Section Faculty:

Dr. Sangeetha Viswanathan

The course owner and the authors of this course deck, Prof. Raja vadhana and Dr.Sangeetha, are gratefully acknowledging all authors who made their course materials freely available online.

☐ Introduction

- ☐ Types of Robots
- ☐ Mobile Robots – Use case of Programmable solutions

☐ Components of Programmable Robotic System

- ☐ Perception
- ☐ Representation
- ☐ Motion

☐ Localization

- ☐ Taxonomy
- ☐ Types

☐ Mapping

- ☐ Feature Based
- ☐ Grid Based

☐ Path Planning

- ☐ Graph Based
- ☐ Sample Based
- ☐ Evolutional Techniques

☐ Human – Robot Interaction

- ☐ Ethics
- ☐ Introduction to Visual Bot
- ☐ Introduction to Chat Bot

Today's Roadmap

01

What is a map frame?

Global vs robot frame, and the translate + rotate math that bridges them.

02

Where does localization show up?

Mobile robots (Roomba, AMR, Mars rover) and software agents (cyber bots, RPA, crawlers).

03

What kind of localization problem is it?

A four-axis taxonomy: static/dynamic, single/multi, local/global, passive/active.

04

How does grid-based localization work?

The histogram filter end-to-end: predict step (motion) and update step (measurement).

05

Where does AI/ML fit?

Noisy sensors, learned models, action recommendation. And when to switch to Monte Carlo.

The Big Question

“Where am I?”

Every autonomous system — physical or digital — must answer this repeatedly, in real time, from noisy and incomplete information.

PERCEPTION

Perception

Sense the world — LIDAR, camera, packet, log.

LOCALIZATION

Localization

Convert sensing into a belief about state.

← *We are here*

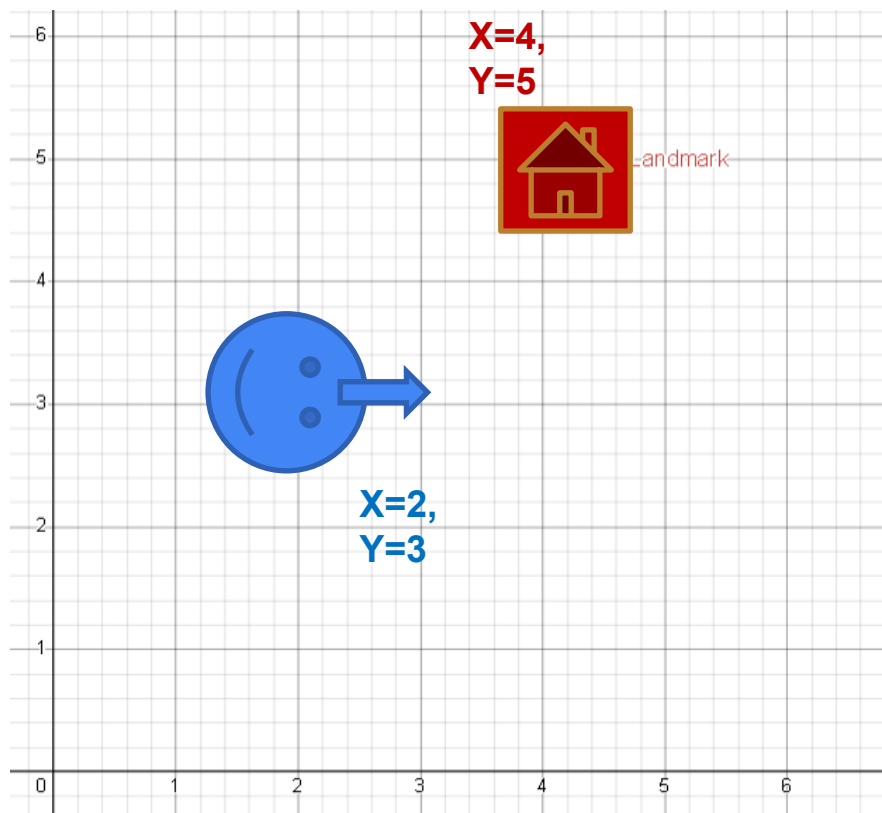
ACTION

Action / Planning

Decide what to do given that belief.

Map Frame

Frame



Global Map Frame

$$Pose_{robot} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix}$$

$$ObjPose_r = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ - \end{bmatrix}$$

Range(r, o) = ~3.16 meters

The Global Map Frame

A fixed origin and axes everyone in the system agrees on

All landmarks, walls, and the robot's pose are expressed in one fixed coordinate system.

Robot pose: $(2, 3)$, $\theta = 45^\circ$

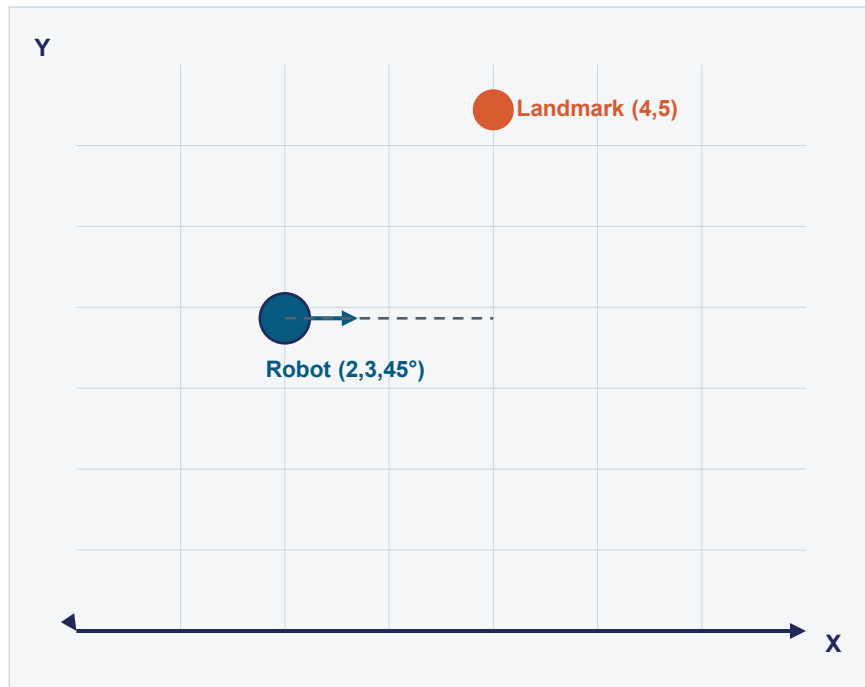
Landmark: $(4, 5)$

Range: $\sqrt{8} \approx 2.83$ m

KEY IDEA

The map frame is fixed. It does not know or care where the robot is — it is the reference the robot localizes against.

Origin is picked at map-creation time (a warehouse corner, a lat-lon, etc.).



The Local Robot Frame

The robot's own system — origin at itself, x-axis along its heading

Everything the robot does — “move forward 1 m”, “scan at +30°”, “wall on my left” — is naturally expressed in this frame.

GLOBAL MAP FRAME

What the map says

Robot at **(2, 3)**, $\theta = 45^\circ$

Landmark at **(4, 5)**

Origin = fixed point on the map

Axes along map north / east

The robot is one object among many here.

LOCAL ROBOT FRAME

What the robot sees

Robot at the **origin (0, 0)**

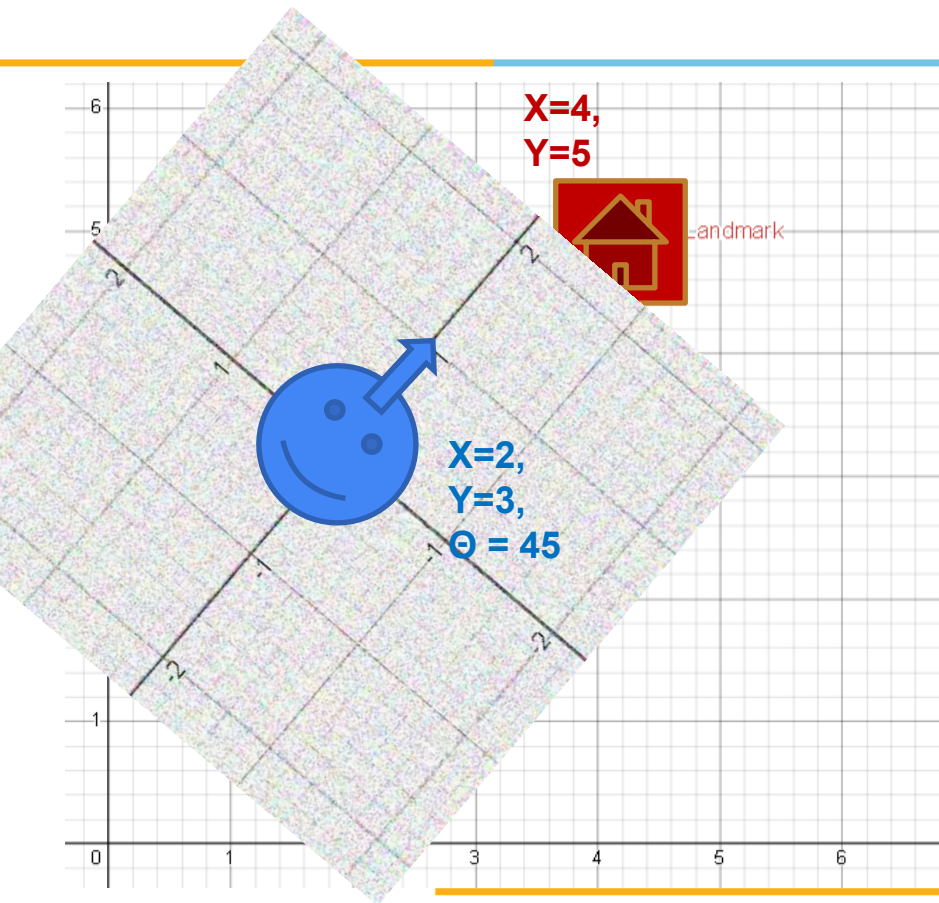
x'-axis = direction it faces

y'-axis = 90° to the left

Landmark = **(2.83, 0)** **straight ahead**

Same landmark, two valid coordinates.

Frame



$$\text{Rot}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$$\text{Rot}(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Local Robot Frame

$$\text{Pose}_{\text{robot}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ \pi/4 \end{bmatrix}$$

$\text{ObjPose}_r = ?$

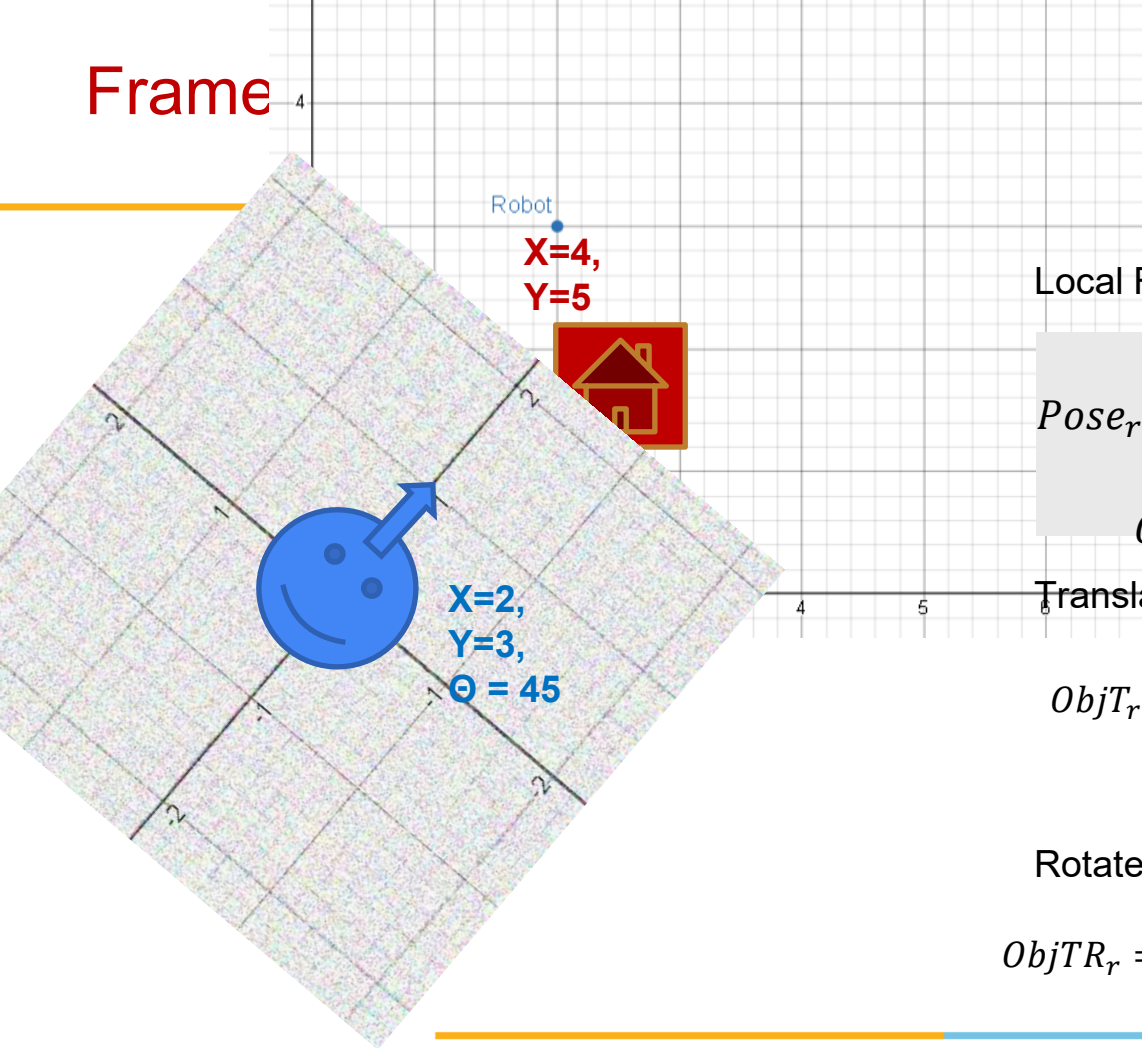
Translate map point to robot frame

$$\text{ObjT}_r = \begin{bmatrix} X - x \\ Y - y \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Rotate coordinates by $-\theta$

$$\text{ObjTR}_r = \begin{bmatrix} X_r \\ Y_r \end{bmatrix} = \begin{bmatrix} \cos\pi/4 & \sin\pi/4 \\ -\sin\pi/4 & \cos\pi/4 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} \sim 2.83 \\ 0 \end{bmatrix}_{10}$$

Frame



$$\text{Rot}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$$\text{Rot}(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Local Robot Frame

$$\text{Pose}_{\text{robot}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ \pi/4 \end{bmatrix}$$

$\text{ObjPose}_r = ?$

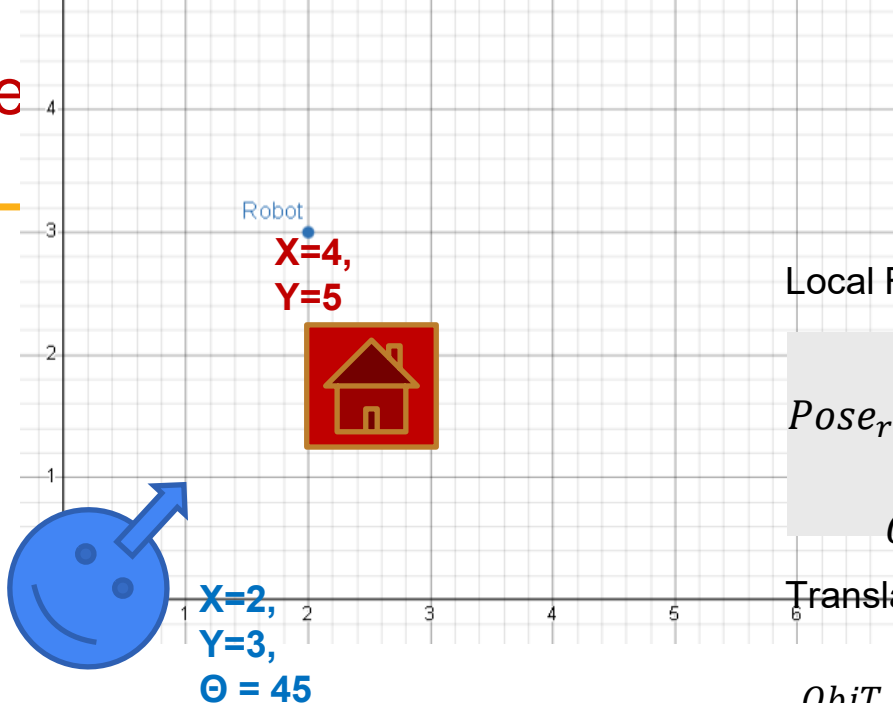
Translate map point to robot frame

$$\text{Obj}T_r = \begin{bmatrix} X - x \\ Y - y \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Rotate coordinates by $-\theta$

$$\text{Obj}TR_r = \begin{bmatrix} X_r \\ Y_r \end{bmatrix} = \begin{bmatrix} \cos\pi/4 & \sin\pi/4 \\ -\sin\pi/4 & \cos\pi/4 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} \sim 2.83 \\ 0 \end{bmatrix}_{11}$$

Frame



$$\text{Rot}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$
$$\text{Rot}(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Local Robot Frame

$$\text{Pose}_{\text{robot}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ \pi/4 \end{bmatrix}$$

$\text{ObjPose}_r = ?$

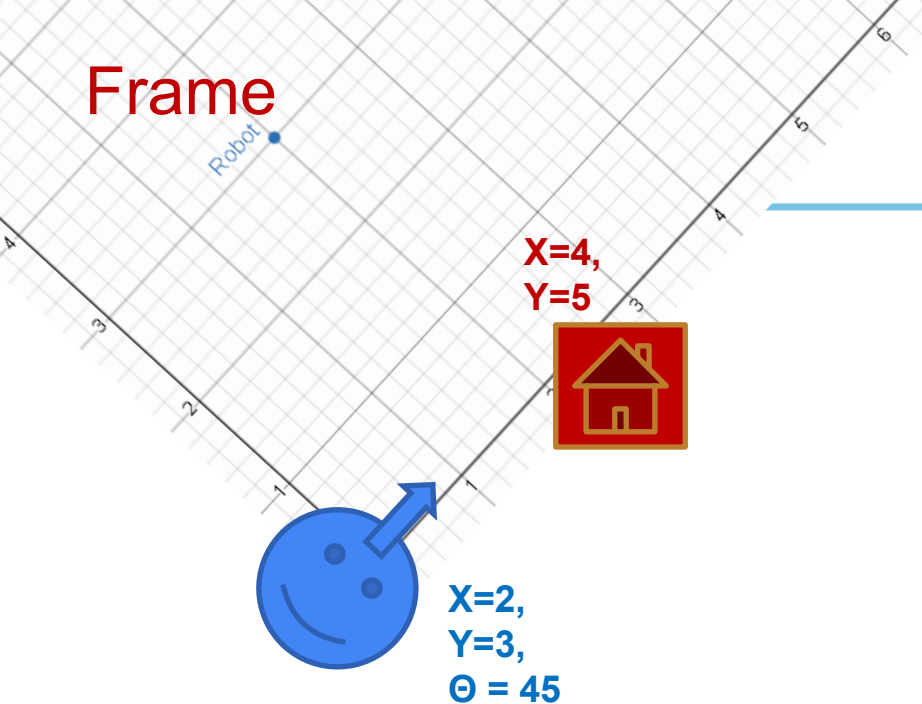
Translate map point to robot frame

$$\text{Obj}T_r = \begin{bmatrix} X - x \\ Y - y \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Rotate coordinates by $-\theta$

$$\text{Obj}TR_r = \begin{bmatrix} X_r \\ Y_r \end{bmatrix} = \begin{bmatrix} \cos\pi/4 & \sin\pi/4 \\ -\sin\pi/4 & \cos\pi/4 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} \sim 2.83 \\ 0 \end{bmatrix}_{12}$$

Frame



$$\text{Rot}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$$\text{Rot}(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Local Robot Frame

$$\text{Pose}_{\text{robot}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ \pi/4 \end{bmatrix}$$

$\text{ObjPose}_r = ?$

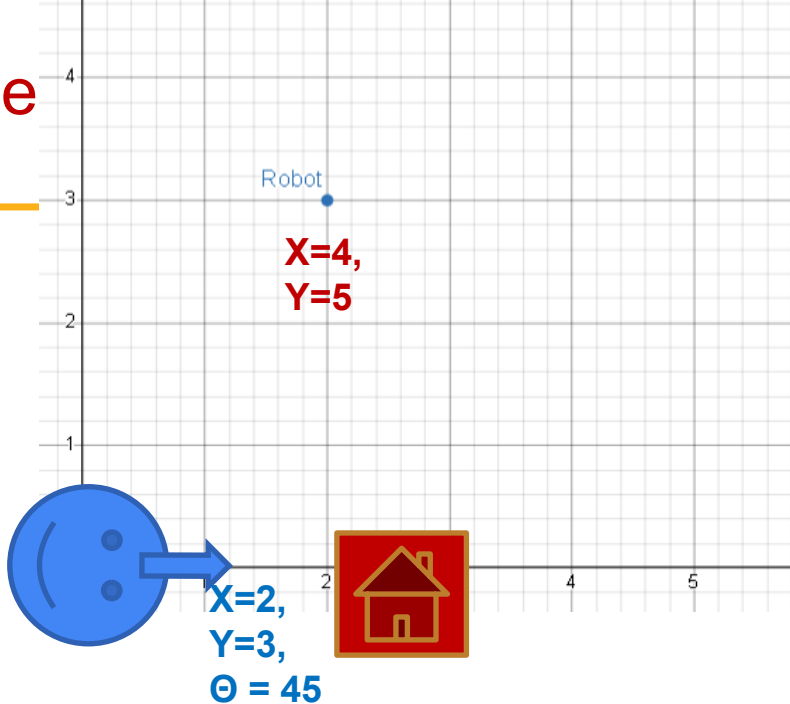
Translate map point to robot frame

$$\text{ObjT}_r = \begin{bmatrix} X - x \\ Y - y \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Rotate coordinates by $-\theta$

$$\text{ObjTR}_r = \begin{bmatrix} X_r \\ Y_r \end{bmatrix} = \begin{bmatrix} \cos\pi/4 & \sin\pi/4 \\ -\sin\pi/4 & \cos\pi/4 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} \sim 2.83 \\ 0 \end{bmatrix}_{13}$$

Frame

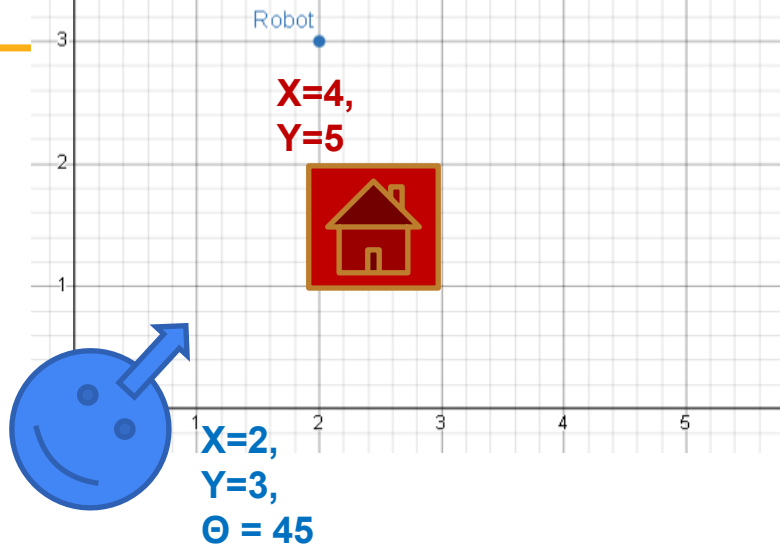


$$\text{Rot}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$
$$\text{Rot}(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Local Robot Frame

$$\text{Pose}_{\text{robot}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$
$$\text{ObjPose}_r = \begin{bmatrix} 2.828 \\ 0 \\ - \end{bmatrix}$$

Frame



Local Robot Frame

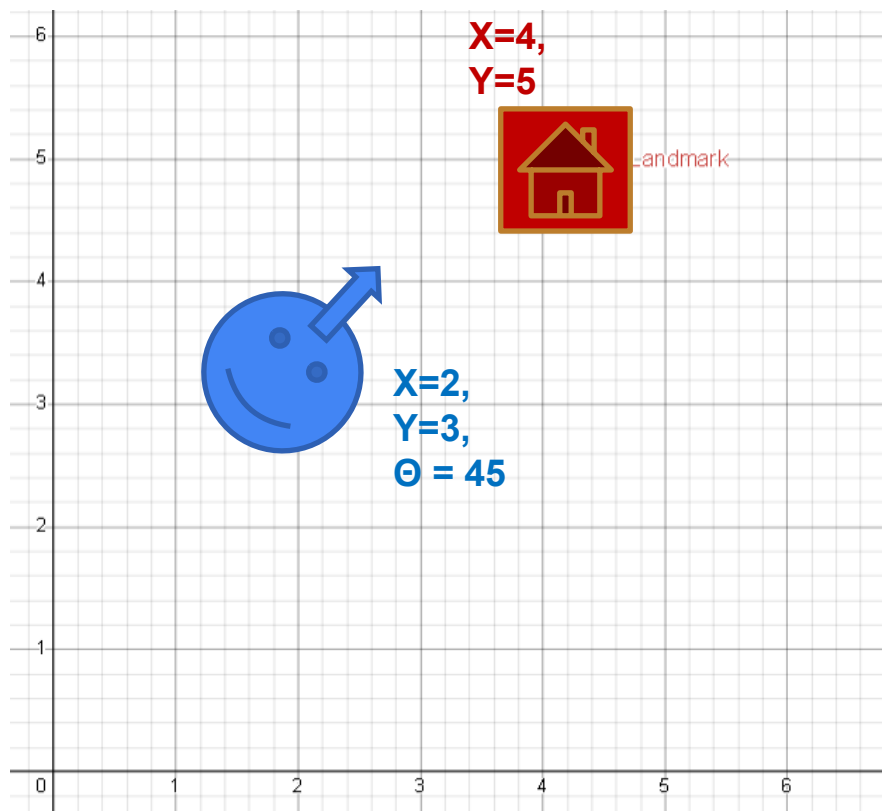
$$Pose_{robot} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$ObjPose_r = \begin{bmatrix} 2.828 \\ 0 \\ - \end{bmatrix}$$

Rotate coordinates by $+\theta$

$$ObjR_m = \begin{bmatrix} X' \\ Y' \end{bmatrix} = \begin{bmatrix} \cos\pi/4 & -\sin\pi/4 \\ +\sin\pi/4 & \cos\pi/4 \end{bmatrix} \begin{bmatrix} 2.838 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Frame



$$\text{Rot}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$$\text{Rot}(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Local Robot Frame

$$\text{Pose}_{\text{robot}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{ObjPose}_r = \begin{bmatrix} 2.828 \\ 0 \\ - \end{bmatrix}$$

Rotate coordinates by $+\theta$

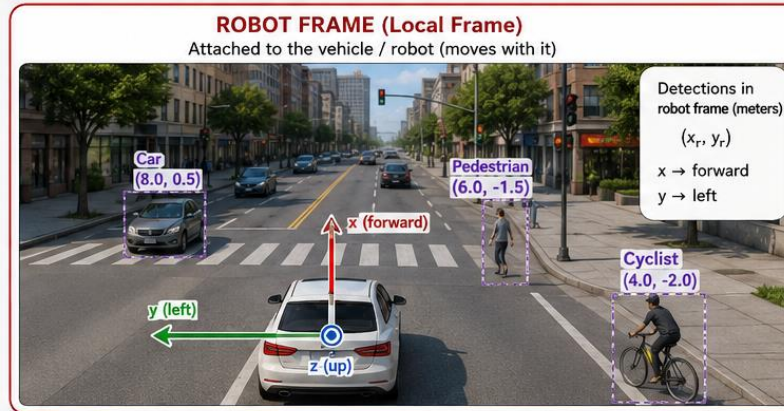
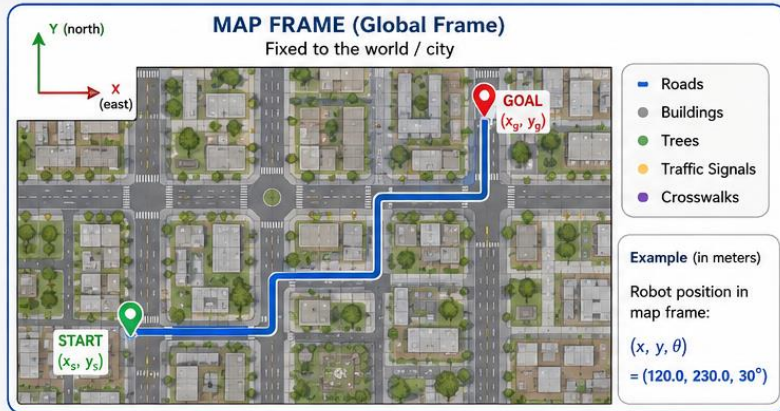
$$\text{ObjR}_m = \begin{bmatrix} X' \\ Y' \end{bmatrix} = \begin{bmatrix} \cos\pi/4 & -\sin\pi/4 \\ +\sin\pi/4 & \cos\pi/4 \end{bmatrix} \begin{bmatrix} 2.838 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Translate robot to map frame

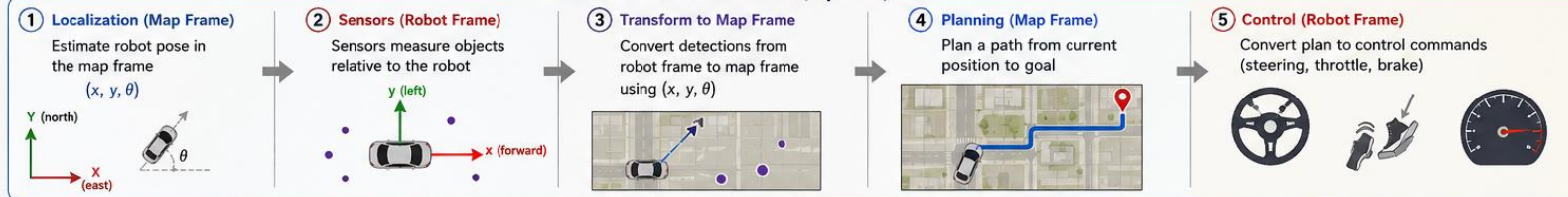
$$\text{ObjRT}_m = \begin{bmatrix} X' + x \\ Y' + y \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

MAP FRAME vs ROBOT FRAME in CITY NAVIGATION

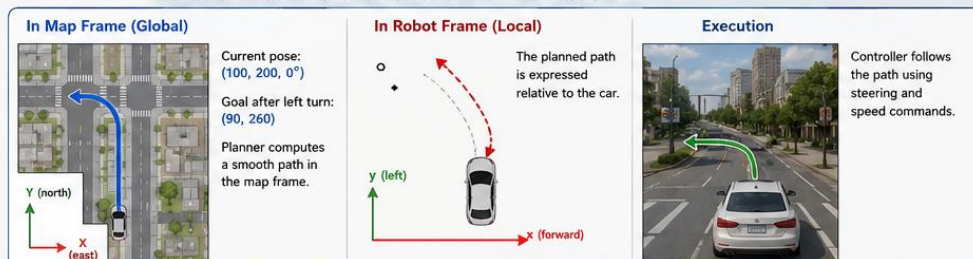
Map Frame = Global (world/city) | **Robot Frame** = Local (attached to robot/car)



HOW THEY WORK TOGETHER (Pipeline)

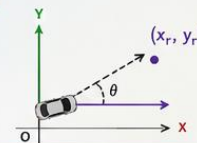


EXAMPLE SCENARIO: LEFT TURN AT AN INTERSECTION



KEY TAKEAWAYS

- Map frame = fixed to the world (city, GPS, HD map)
- Robot frame = attached to the robot (moves with it)
- Sensors give data in robot frame
- Planning happens in map frame
- Control/actuation happens in robot frame
- Transformation between frames is done using the robot pose (x, y, θ)



TRANSFORMATION (2D)

Convert a point from robot frame (x_r, y_r) to map frame (x_m, y_m)

$$x_m = x + x_r \cos \theta - y_r \sin \theta$$

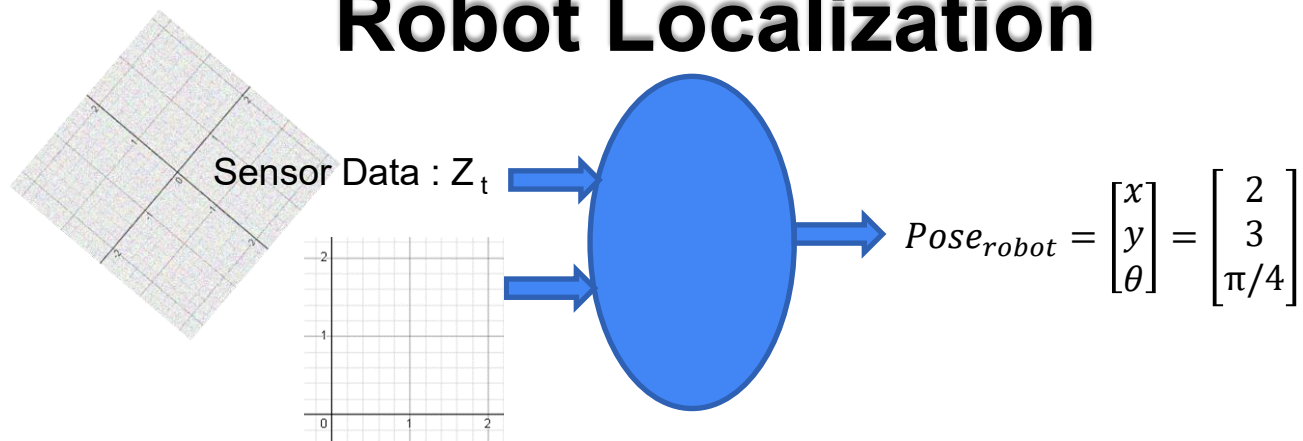
$$y_m = y + x_r \sin \theta + y_r \cos \theta$$

Where (x, y, θ) is the robot pose in the map frame.

MAP FRAME = Where things are in the world

ROBOT FRAME = Where things are relative to the robot

Robot Localization



Robot Localization — Formally

A continuous estimation problem, not a one-shot fix

DEFINITION

Localization is the problem of maintaining a probability distribution over the robot's pose, given a map, a sequence of control inputs, and a sequence of sensor measurements.

Inputs

Map (m)

World model — 2D grid, point cloud, topological graph.

Controls ($u_{1:t}$)

Control inputs — wheel velocities, steering, accelerations.

Measurements ($z_{1:t}$)

Sensor readings — LIDAR, camera, IMU, GPS.

Bayes filter

predict + update

Output

$bel(x_t)$

Belief — a probability distribution over all possible poses at time t .

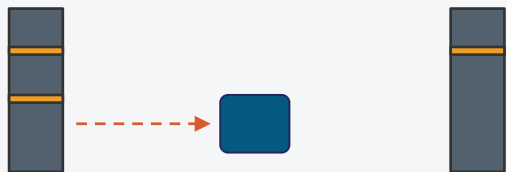
Not a single number — a distribution. “I’m 70% sure I’m here, 25% over there, 5% scattered.”

Use Cases — Mobile Robots

State = (x, y, θ) . Sensors = LIDAR, camera, IMU, odometry, GPS.

Warehouse AMR

Finds itself in the pallet aisle.



Roomba / vacuum

Knows which rooms it cleaned.



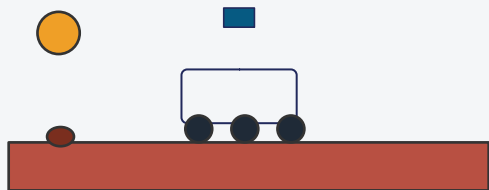
Self-driving car

Lane-level pose vs HD map.



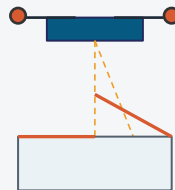
Mars rover

Visual-inertial pose, no GPS.



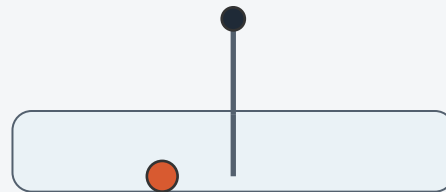
Delivery drone

Geo-pose for last-100m landing.



Surgical robot

Tool-tip pose vs anatomy.

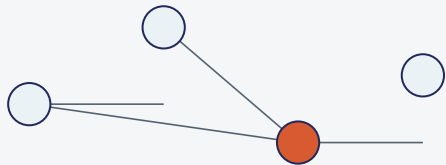


Use Cases — Software Agents

State = (host, subnet, privilege). “Sensors” = packets, syscalls, logs.

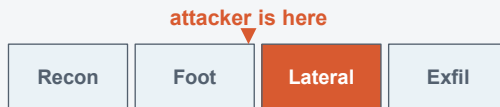
Pen-test bot

Locates itself in the net graph.



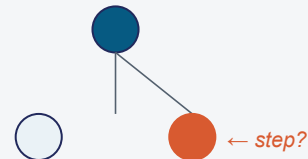
Threat-hunter

Pinpoints attacker's stage.



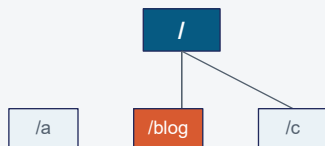
SOAR bot

Finds the kill-chain stage.



Web crawler

Its position in the site graph.



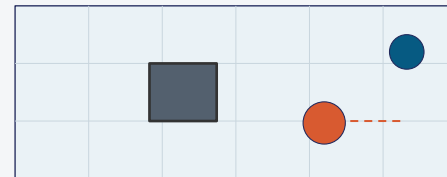
RPA bot

Its place in a workflow.



Game NPC

Pose in a virtual world.



The Shared Bayes-Filter Pattern

Physical or digital, the math is the same — only the state changes

THE BAYES-FILTER PATTERN

$$\text{bel}(x_t) = \eta \cdot p(z_t \mid x_t) \cdot \int p(x_t \mid u_t, x_{t-1}) \cdot \text{bel}(x_{t-1}) \, dx_{t-1}$$

Predict (motion) → Update (measurement) → Normalize → Loop.

Component	Physical (mobile robot)	Logical (software agent)
State (x_t)	Pose: (x, y, θ)	Network position: (host, subnet, privilege)
Action (u_t)	Move forward 1 m, turn 30°	scp file, sudo, click button
Sensor (z_t)	LIDAR ranges, image, IMU	Packet capture, log line, API response
Goal	Reach pose (xg, yg)	Reach asset (DB server, admin shell)

“Where am I?” is the same question. The substrate changes; the algorithm doesn’t.

Taxonomy of Localization Problems

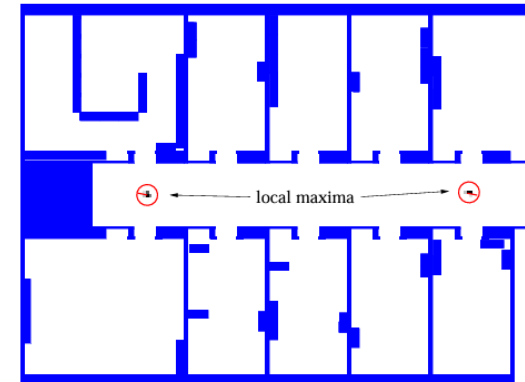
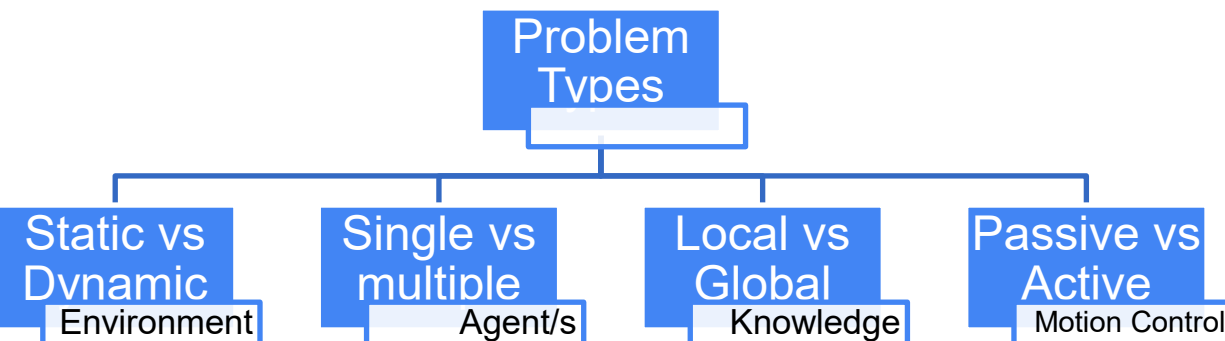
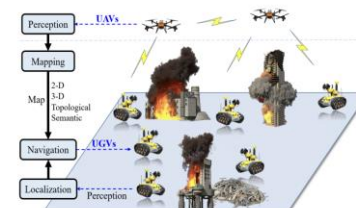


Figure 7.2 Example situation that shows a typical belief state during global localization in a locally symmetric environment. The robot has to move into one of the rooms to determine its location.



- Varying Features of the Environment
- Knowledge of Initial Pose
- Level of Uncertainty
- Recovery from Failures : Kidnapped Robot Problem
- Necessity for Real time Control

Four Axes of Localization Problems

Pick a side on each axis — the combination tells you which algorithm applies

Environment	STATIC Walls and landmarks don't move.	vs	DYNAMIC People, vehicles, objects move.
Agents	SINGLE One robot, one belief.	vs	MULTIPLE Robots share or compete for a map.
Knowledge	LOCAL (tracking) We know roughly where we started.	vs	GLOBAL (kidnapped) No initial pose — could be anywhere.
Motion Control	PASSIVE Localize from whatever motion happens.	vs	ACTIVE Move on purpose to cut uncertainty.

Example: a Roomba in a quiet room is STATIC · SINGLE · LOCAL · PASSIVE → textbook grid filter works.

Axis 1 — Static vs Dynamic Environment

Does anything in the map move while the robot localizes?

STATIC

Map features don't change

Walls, landmarks and obstacles are fixed across the localization window.

EXAMPLES

- Roomba in an empty room
- Warehouse AMR after hours
- Mars rover (Mars doesn't move)

DYNAMIC

Features change over time

People walk through, vehicles move, the attacker pivots between hosts.

EXAMPLES

- Self-driving car in traffic
- Service robot in an airport
- Threat-hunter chasing a target

Axis 2 — Single vs Multiple Agents

Are we localizing one entity, or many that can share information?

SINGLE AGENT

One robot, one belief

All information comes from this robot's own sensors and motion history.

EXAMPLES

- Single Roomba in a house
- Solo Mars rover
- One pen-test bot on a target

MULTIPLE AGENTS

Several robots, shared map

Robots observe each other and share beliefs — cooperative localization.

EXAMPLES

- Warehouse fleet (Amazon, Ocado)
- Drone swarm in search & rescue
- Connected vehicles (V2V)

WHY HARDER

Joint belief grows exponentially with the number of agents — needs decentralized algorithms.

Axis 3 — Local vs Global Knowledge

How well do we know the initial pose? Can we recover if moved?

LOCAL (tracking)

We know where we started

Belief is a tight unimodal distribution; the filter tracks small corrections.

EXAMPLES

- Roomba returning to dock
- Lane keeping in a car
- Arm tracking from a zero

GLOBAL (kidnapped)

Could be anywhere

Belief starts uniform — every cell equally likely; converge from scratch.

EXAMPLES

- Roomba just turned on
- Robot picked up and moved
- Bot dropped in unknown subnet

KIDNAPPED-ROBOT PROBLEM

A confident robot is teleported. Grid filters keep mass everywhere and can recover; pure Kalman filters collapse to one mode and stay wrong.

Axis 4 — Passive vs Active Localization

Do we choose motions to help ourselves localize?

PASSIVE

Localize from whatever happens

Motion is dictated by the task; the filter just reads u and z .

EXAMPLES

- Roomba runs its clean pattern
- Car drives the chosen route
- Crawler follows existing links

ACTIVE

Move to reduce uncertainty

Pick the action with the most expected information gain — go disambiguate.

EXAMPLES

- Drive to a corner to break symmetry
- Rover picks informative viewpoint
- Active scanning to ID the OS

WHERE RL SHINES

Active localization learns a policy that minimises belief entropy — a major entry point for reinforcement learning.

Where AI/ML Lives Inside Localization

Five entry points into the classical Bayes pipeline

Component	Why classical struggles	Where AI/ML helps
Motion model $p(x_t u_t, x_{t-1})$	Wheels slip, terrain varies.	Learn noise from telemetry (GP regression).
Measurement model $p(z_t x_t)$	Sensor responses non-Gaussian.	Neural likelihood — CNN(image, pose) → score.
Map representation	Hand-built maps go stale.	Learned features (SuperPoint); NeRF / occupancy fields.
State estimator itself	Real distributions are weird.	DeepKF / deep variational filters learn the update.
Active control policy	Hand-tuned heuristics fall short.	RL — reward = information gain.

ML wins where the analytical model breaks down — unmodellable terrain, non-Gaussian sensors.

Discrete Bayes Filters



$$Bel(x_t) = \eta \boxed{P(z_t|x_t)} \int_{x_{t-1}} \boxed{P(x_t|u_t, x_{t-1})} Bel(x_{t-1}) dx_{t-1}$$

 Perception Model  Motion Model

Recursive State Estimation

Bayes Filter Algorithm

1. Initial Belief
2. Control Update
3. Measurement Update

X_{t-1}	U_t	$P(X_t = \text{is_open})$	$P(X_t = \text{is_closed})$
is_open	push	1	0
is_closed	Push	0.8	0.2
is_open	do_nothing	1	0
is_closed	do_nothing	0	1

X_0	$P(X_0)$
is_open	0.5
is_closed	0.5

X_t	$P(Z_t = \text{sense_open})$	$P(Z_t = \text{sense_closed})$
is_open	0.6	0.4
is_closed	0.2	0.8

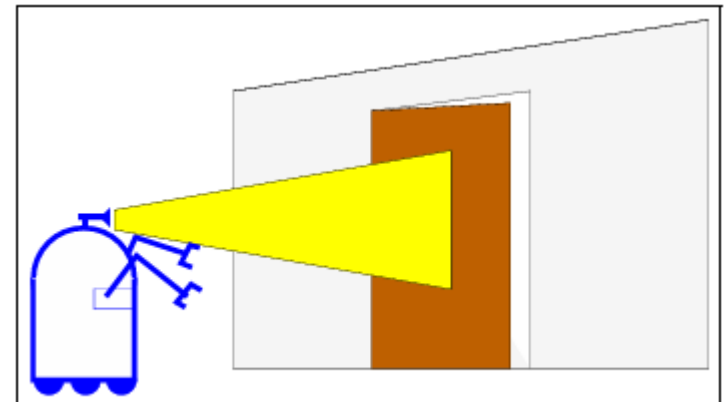


Figure 2.2 A mobile robot estimating the state of a door.

Histogram Filter Approach

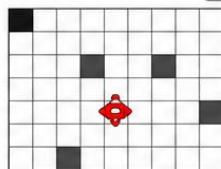
$$Bel(x_t) = \eta \boxed{P(z_t|x_t)} \int_{x_{t-1}} \boxed{P(x_t|u_t, x_{t-1})} Bel(x_{t-1}) dx_{t-1}$$

Perception Model Motion Model

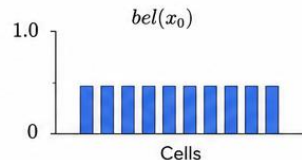
1. GRID LOCALIZATION – ALGORITHM FLOW (Thrun)

The robot maintains a belief (probability) over all grid cells.
At each time step: predict with the motion model, update with the measurement, then normalize.



INITIAL BELIEF

$bel(x_0)$: probability distribution over all grid cells (usually uniform if no information)

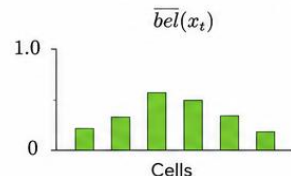


1) PREDICTION (MOTION UPDATE)

Use motion model $p(x_t | u_t, x_{t-1})$ to predict belief after action u_t .

$$\bar{bel}(x_t) = \sum_{x_{t-1}} p(x_t | u_t, x_{t-1}) bel(x_{t-1})$$

Effect: probability mass spreads according to motion uncertainty.

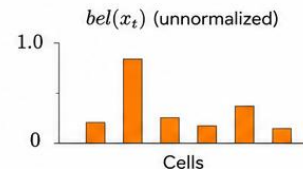


2) CORRECTION (MEASUREMENT UPDATE)

Incorporate sensor measurement z_t using measurement model $p(z_t | x_t)$.

$$bel(x_t) = \eta p(z_t | x_t) \bar{bel}(x_t)$$

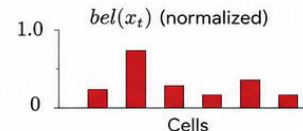
Effect: increases probability where observation is likely.



3) NORMALIZE

Make the belief a valid probability distribution (sum to 1).

$$bel(x_t) \leftarrow \frac{bel(x_t)}{\sum_x bel(x_t)}$$



OUTPUT

Current belief $bel(x_t)$ over all cells.
Repeat for next time step $t + 1$.



```

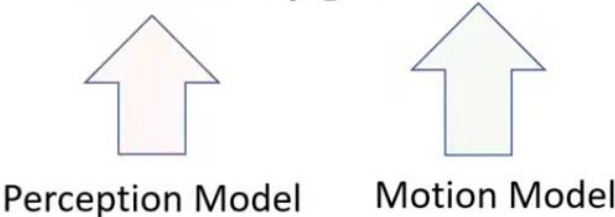
1: Algorithm Grid_Localization( $\{p_{k,t-1}\}, u_t, z_t, m$ ):
2:   for all  $k$  do
3:      $\bar{p}_{k,t} = \sum_i p_{i,t-1} \text{motion\_model}(\text{mean}(\mathbf{x}_k), u_t, \text{mean}(\mathbf{x}_i))$ 
4:      $p_{k,t} = \eta \text{measurement\_model}(z_t, \text{mean}(\mathbf{x}_k), m)$ 
5:   endfor
6:   return  $\{p_{k,t}\}$ 
    
```

Grid Based Localization

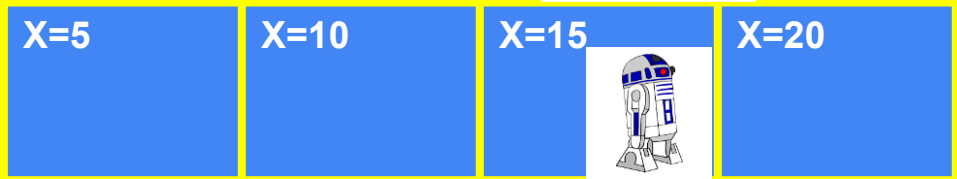


Histogram Filter Approach

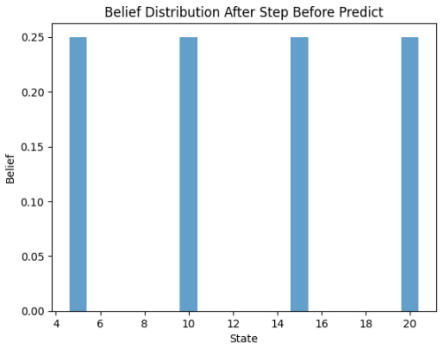
$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$



X_0	$P(X_0)$
5	0.25
10	0.25
15	0.25
20	0.25



X_t	$P(Z_t=5)$	$P(Z_t=10)$	$P(Z_t=15)$	$P(Z_t=20)$
5	0.8	0.1	0.1	0
10	0.3	0.4	0.2	0.1
15	0.1	0.2	0.4	0.3
20	0	0.1	0.1	0.8



Grid Based Localization

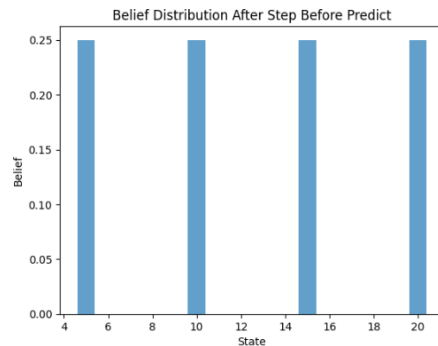
Histogram Filter

$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

Perception Model

Motion Model

Predict Step:
(Motion)
Update Step:
(Measurement)



X_t	$P(Z_t=15)$	X_0	$P(X_0)$
5	0.1	5	0.25
10	0.2	10	0.25
15	0.4	15	0.25
20	0.1	20	0.25

X=5

X=10

X=15



X=20

X_{t-1}	U_t	$P(X_t=5)$	$P(X_t=10)$	$P(X_t=15)$	$P(X_t=20)$
5	→	0.2	0.6	0.2	0
10	→	0.3	0.1	0.6	0.1
15	→	0.1	0.3	0.1	0.5
20	→	0	0.1	0.3	0.6

Grid Based Localization



Histogram Filter

$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

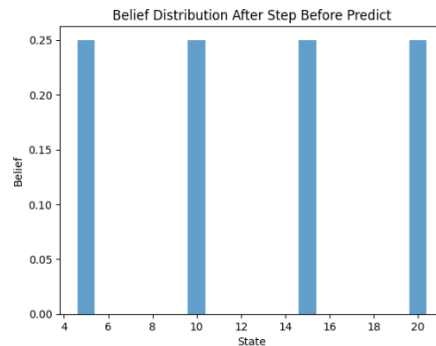


Perception Model

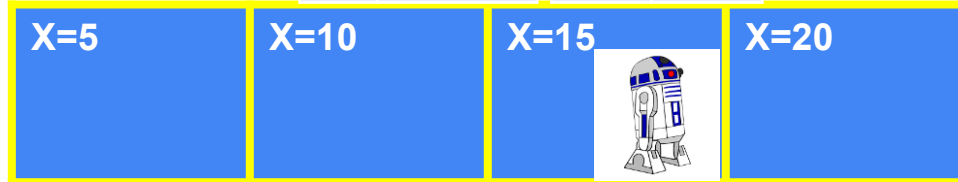


Motion Model

Predict Step:



X_t	$P(Z_t=15)$	X_0	$P(X_0)$
5	0.1	5	0.25
10	0.2	10	0.25
15	0.4	15	0.25
20	0.1	20	0.25



X_{t-1}	U_t	$P(X_t=5)$	$P(X_t=10)$	$P(X_t=15)$	$P(X_t=20)$
5	→	0.2	0.6	0.2	0
10	→	0.3	0.1	0.6	0.1
15	→	0.1	0.3	0.1	0.5
20	→	0	0.1	0.3	0.6

Grid Based Localization

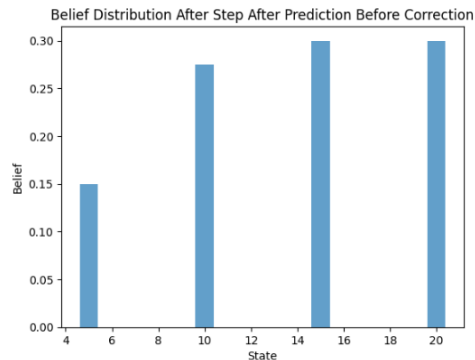
Histogram Filter

$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

Perception Model

Motion Model

Predict Step:



X_t	$P(Z_t=15)$	X_0	$P(X_0)$
5	0.1	5	0.25
10	0.2	10	0.25
15	0.4	15	0.25
20	0.1	20	0.25

X=5

X=10

X=15



X=20

X_{t-1}	U_t	$P(X_t=5)$	$P(X_t=10)$	$P(X_t=15)$	$P(X_t=20)$
*	→	0.15	0.275	0.3	0.3

Grid Based Localization



Histogram Filter

$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

Perception Model

Motion Model

X_t	$P(Z_t=15)$	X_0	$P(X_0)$
5	0.1	5	0.25
10	0.2	10	0.25
15	0.4	15	0.25
20	0.1	20	0.25

X=5

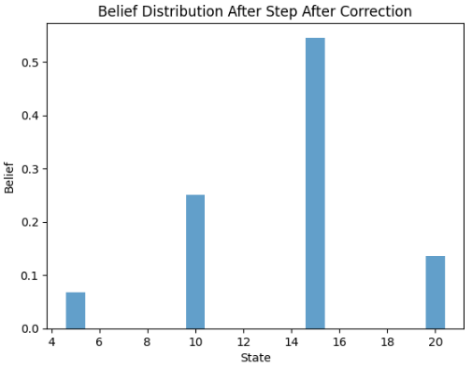
X=10

X=15

X=20

X_{t-1}	U_t	$P(X_t=5)$	$P(X_t=10)$	$P(X_t=15)$	$P(X_t=20)$
*	→	0.15	0.275	0.3	0.3
		0.015	0.055	0.12	0.03
		0.06	0.25	0.55	0.14

Update Step:



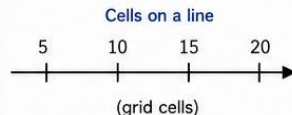
2. EXAMPLE PROBLEM FLOW – DETAILED WORKING

States (grid cells): $x \in \{5, 10, 15, 20\}$

Initial belief: $bel(x_0) = [0.25, 0.25, 0.25, 0.25]$

Action u_t : move right (one step)

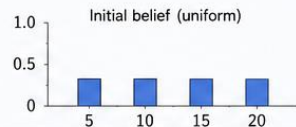
Measurement z_t : 15



B. FILTERING STEPS

0) Initial belief $bel(x_0)$

x	5	10	15	20
$bel(x_0)$	0.25	0.25	0.25	0.25



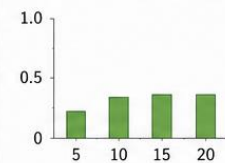
1) PREDICTION (Motion Update)

$$\bar{bel}(x_t) = \sum_{x_{t-1}} p(x_t | u_t, x_{t-1}) bel(x_{t-1})$$

To get belief of each current cell x_t , we sum contributions from all previous cells x_{t-1} .

From x_{t-1}	$bel(x_{t-1})$	To $x_t \rightarrow$			
		5	10	15	20
5	0.25	0.05	0.15	0.05	0.00
10	0.25	0.075	0.025	0.150	0.025
15	0.25	0.025	0.075	0.025	0.125
20	0.25	0.000	0.025	0.075	0.150
$\bar{bel}(x_t)$		0.15	0.275	0.30	0.30

Predicted belief $\bar{bel}(x_t)$



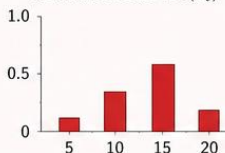
Explanation (example): To be in cell 5 now, robot could have come from cell 5 (0.05), from cell 10 (0.075), from cell 15 (0.025), from cell 20 (0.0). Sum = 0.15.

3) NORMALIZE

Sum of unnormalized values = $0.015 + 0.055 + 0.12 + 0.03 = 0.22$

x_t	5	10	15	20
Unnormalized	0.015	0.055	0.120	0.030
Normalized $bel(x_t)$	$0.015/0.22$	$0.055/0.22$	$0.120/0.22$	$0.030/0.22$
=	0.06	0.25	0.55	0.14

Normalized belief $bel(x_t)$



Normalize so that $\sum_x bel(x_t) = 1$.

A. MODELS

1) MOTION MODEL $p(x_t | u_t, x_{t-1})$
(move right by 1 cell)

From x_{t-1} - To x_t	5	10	15	20
5	0.2	0.6	0.2	0
10	0.3	0.1	0.6	0.1
15	0.1	0.3	0.1	0.5
20	0	0.1	0.3	0.6

2) MEASUREMENT MODEL $p(z_t = 15 | x_t)$

x_t	5	10	15	20
$p(z = 15 x_t)$	0.1	0.2	0.4	0.1

Sensor says the observed value is $z_t = 15$.

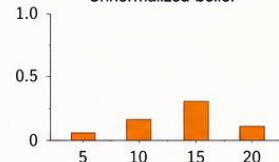
2) CORRECTION (Measurement Update)

$$bel(x_t) \propto \bar{bel}(x_t) p(z_t = 15 | x_t)$$

x_t	$\bar{bel}(x_t)$	$p(z = 15 x_t)$	Product (unnormalized)
5	0.15	0.1	0.015
10	0.275	0.2	0.055
15	0.30	0.4	0.120
20	0.30	0.1	0.030

We multiply predicted belief with measurement likelihood. Cells more consistent with $z_t = 15$ get higher weight.

Unnormalized belief

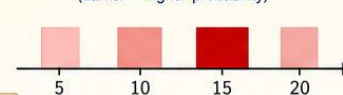


RESULT

After one prediction and one measurement update ($z_t = 15$), the belief distribution is:

x	5	10	15	20
$bel(x_t)$	0.06	0.25	0.55	0.14

Belief as heat map on the line
(darker = higher probability)



The robot is most likely in cell 15 (probability = 0.55), but there is still uncertainty over other cells.

2x2 GRID WORLD



1	2
3	4

Cells: {1, 2, 3, 4}

Start belief (initial):

$bel_0 = [0.25, 0.25, 0.25, 0.25]$

Notation

$bel(x)$ = belief in cell x

We list beliefs in order
[1, 2, 3, 4]

Grid layout:

1	2
3	4

STEP 1: ACTION = MOVE UP

(Robot tries to move up)

PREDICT (after move UP)

Motion model (intuitive)

- 0.7 go in intended direction
- 0.2 stay
- 0.1 go to left/right (if possible)

Predicted belief $\bar{bel}_1$

0.225	0.225
0.050	0.050

As a vector:

[0.225, 0.225, 0.050, 0.050]

UPDATE with measurement

Sensor reads: z_1 = "near wall"

Sensor model $P(z_1 | x)$

0.80	0.80
0.30	0.30

Unnormalized belief

0.180	0.180
0.015	0.015

Normalize (sum = 0.39)

Updated belief bel_1

0.462	0.462
0.038	0.038

As a vector:

[0.462, 0.462, 0.038, 0.038]

Belief heatmap after Step 1 (bel_1)



Most of the belief is now in the top row (as expected after moving UP).

STEP 2: ACTION = MOVE LEFT

(Robot tries to move left)

Current belief (bel_1)

0.462	0.462
0.038	0.038

As a vector:

[0.462, 0.462, 0.038, 0.058]

PREDICT (after move LEFT)

Motion model (intuitive)

- 0.7 go in intended direction
- 0.2 stay
- 0.1 go up/down (if possible)

Predicted belief $\bar{bel}_2$

0.414	0.092
0.036	0.008

As a vector:

[0.414, 0.092, 0.036, 0.008]

UPDATE with measurement

Sensor reads: z_2 = "corner"

Sensor model $P(z_2 | x)$

0.90	0.20
0.20	0.10

Unnormalized belief

0.3726	0.0184
0.0072	0.0008

Normalize (sum = 0.399)

Updated belief bel_2

0.933	0.046
0.018	0.002

As a vector:

[0.933, 0.046, 0.018, 0.002]

Belief heatmap after Step 2 (bel_2)



Now almost all belief is in cell 1 (top-left corner).

SUMMARY OF THE TWO STEPS

- 1) Move UP + measurement "near wall" → belief concentrated in top row.
- 2) Move LEFT + measurement "corner" → belief concentrated in top-left cell (1).

Legend (cells)

1	2
3	4

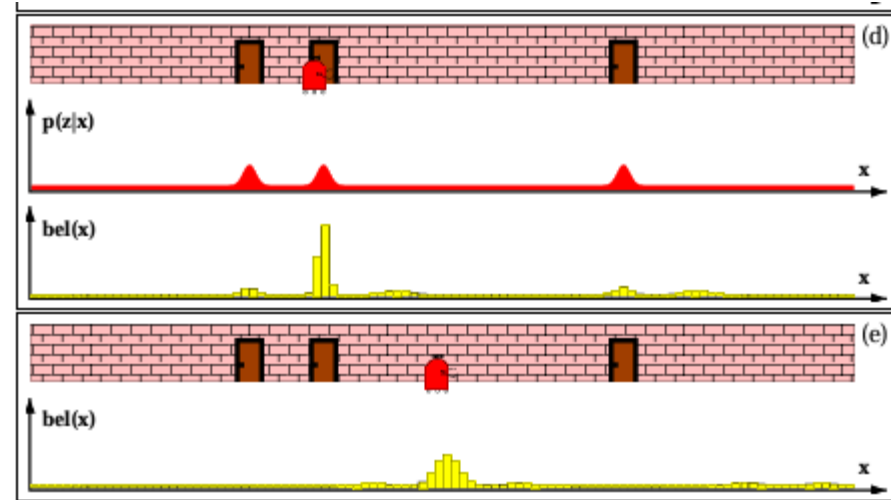
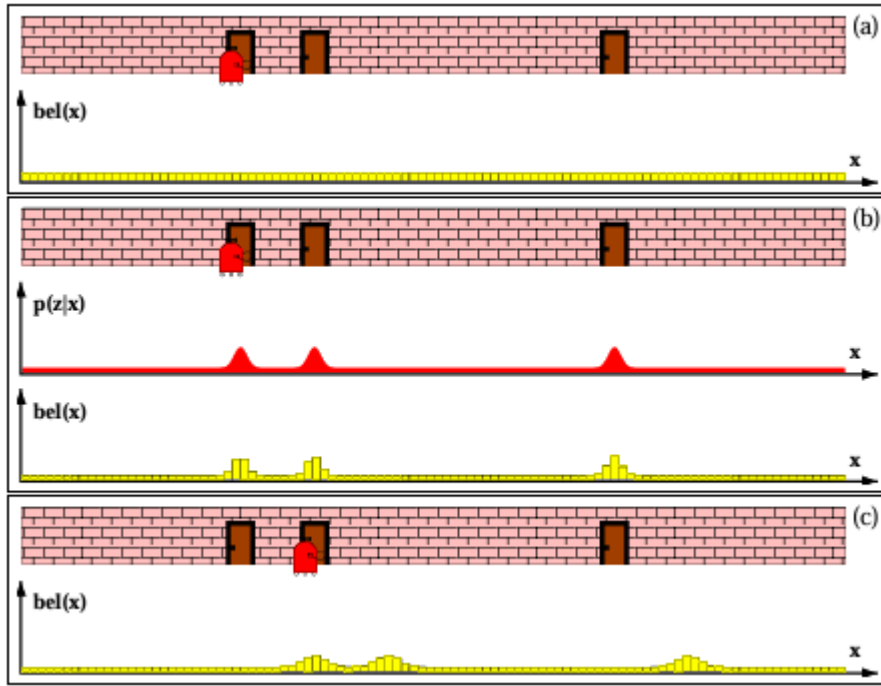
1: top-left
2: top-right
3: bottom-left
4: bottom-right

Key Takeaway

Prediction (motion) spreads and shifts belief.
Update (measurement) sharpens belief.
Repeat at every time step!



Histogram Filter Approach



Grid Based Localization :



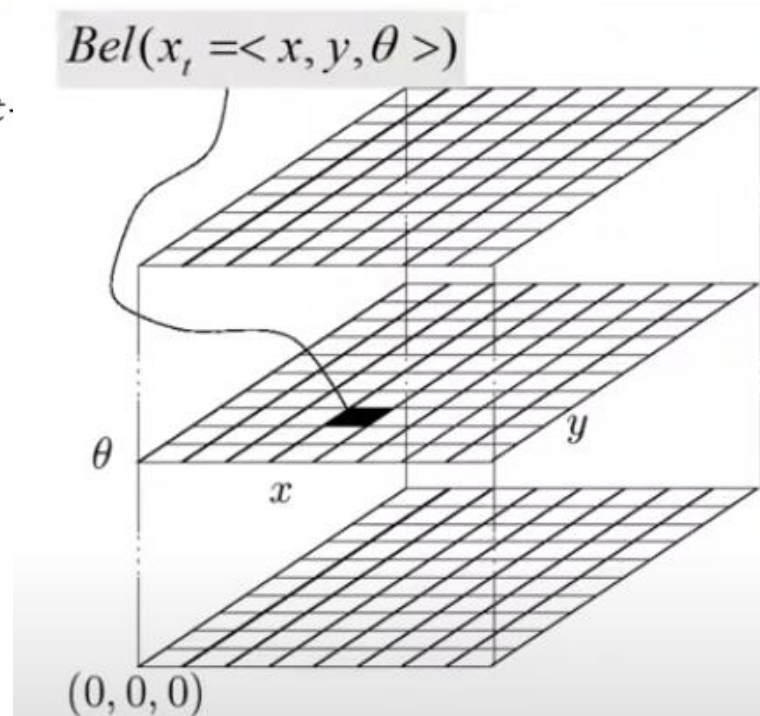
Piecewise Constant Approximation

$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_t.$$

↑
Perception Model

↑
Motion Model

Accuracy Vs Resource Intensive

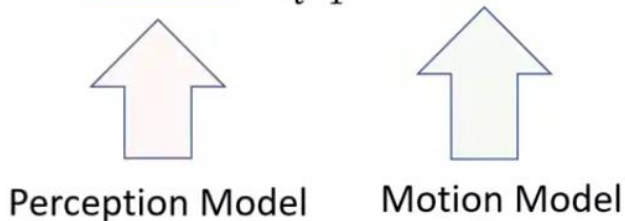


Grid Based Localization :

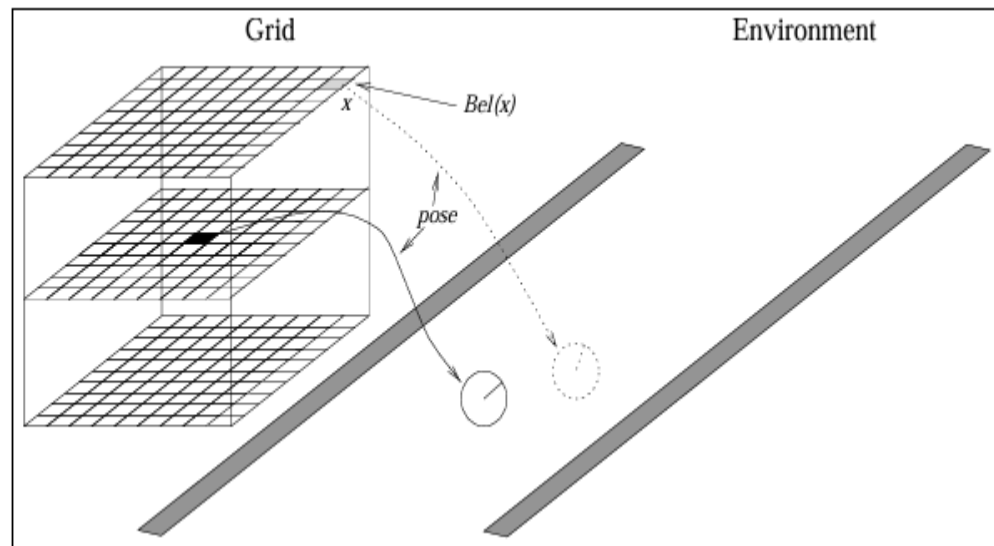


Optimization Techniques

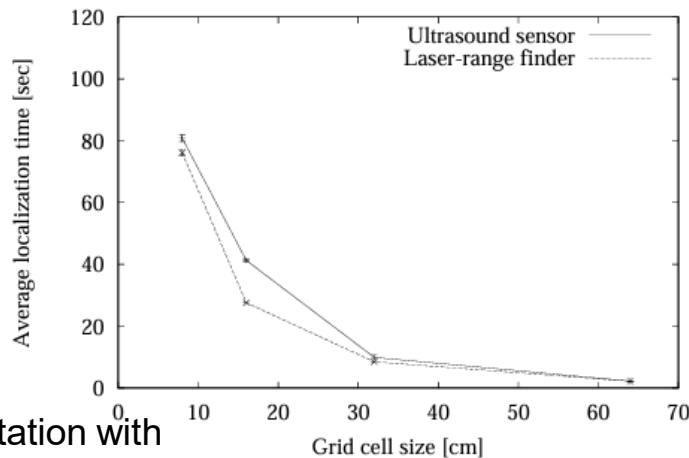
$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$



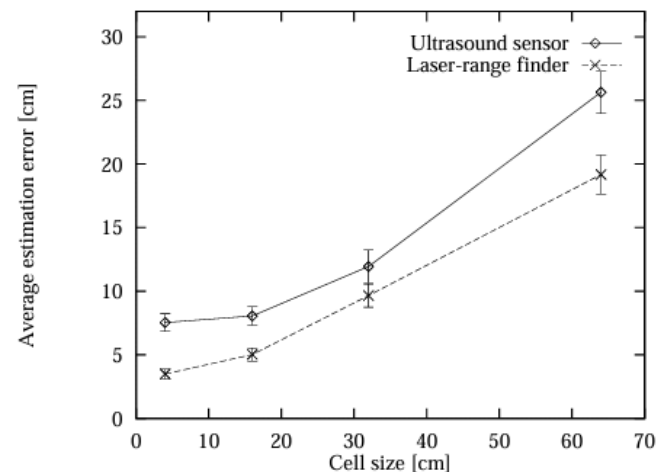
Fine Grained Representation with fixed Grid Structures



Optimization Techniques



Fine Grained Representation with fixed Grid Structures



AIML in Grid Localization :



Use Cases:

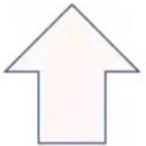
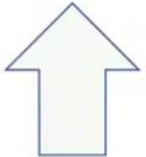
1. Multiple Sensor Input are available . Measurement Models are noisy.
2. Image Captured from camera as streams
3. Recommend Actions for localization

Grid Based Localization :



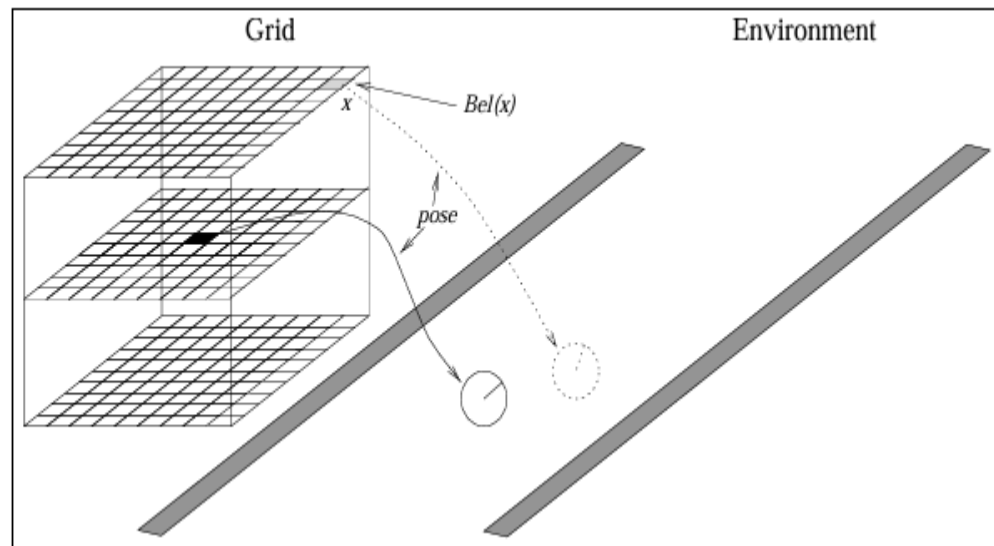
Optimization Techniques

$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

 Perception Model  Motion Model

Fine Grained Representation with fixed Grid Structures

- Pre-caching
- Sensor Sub Sampling
- Delayed Motion Updates
- Selective Updating
- Heuristics



Grid Based Localization :



Optimization Techniques

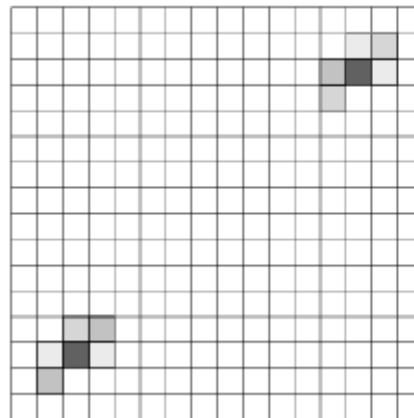
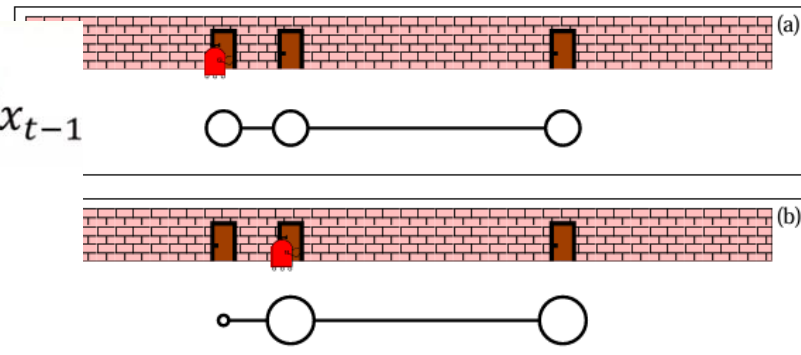
$$Bel(x_t) = \eta P(z_t|x_t) \int_{x_{t-1}} P(x_t|u_t, x_{t-1}) Bel(x_{t-1}) dx_{t-1}$$

Perception Model

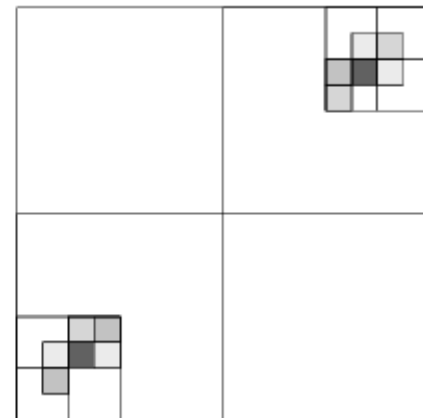
Motion Model

Coarse Grained Representation
with Dynamic Grid Structures

- Topological Representation
- Inflate Motion Noise
- Inflate Measurement Noise



(a)



(b)

Grid Based Localization :

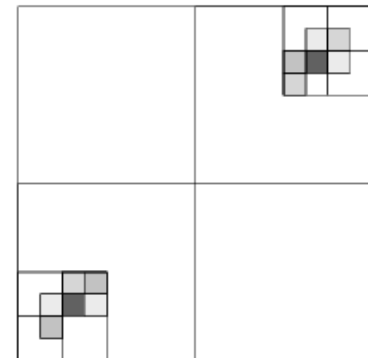


Optimization Techniques

Time	Pose	Grid	Bel(0)	Bel(1)	
0	0.0	0	1.0	0	
1	0.2	0	1.0	0	
2	0.4	0	1.0	0	
3	0.6	0	1.0	0	
4	0.8	0	1.0	0	
5	1.0	1	0	1.0	

Coarse Grained Representation with Dynamic Grid Structures

- Topological Representation
- **Inflate Motion Noise**
- Inflate Measurement Noise
 - 1m Grid
 - Robot moves right to 0.2m from $x=0$
 - Landmark at 3m



(b)

Grid Based Localization :



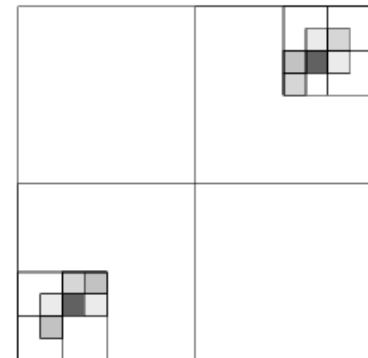
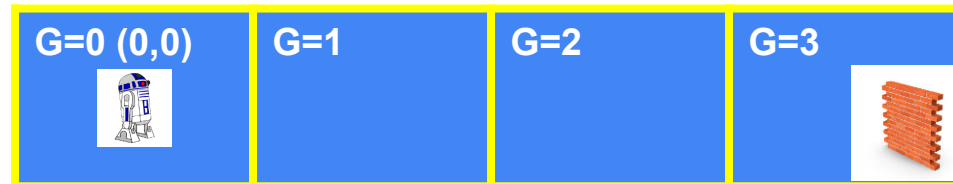
Optimization Techniques

Time	Pose	Grid	Bel(0)	Bel(1)	
0	0.0	0	1.0	0	
1	0.2	0	0.8	0.2	
2	0.4	0	0.64	0.36	
3	0.6	0	0.512	0.488	
4	0.8	0	0.41	0.59	
5	1.0	1	0.33	0.67	

Coarse Grained Representation with Dynamic Grid Structures

- Topological Representation
- **Inflate Motion Noise**
- Inflate Measurement Noise
 - 1m Grid
 - Robot moves right to 0.2m from $x=0$
 - Landmark at 3m

Add Noise to Motion by adding small probability proportional to movement / cellSize = 20%



(b)

Grid Based Localization :



Optimization Techniques

$P(Z | X) = 0.0003$ (Apply Gaussian Dist)

Inference :

Most likely the robot is not in Grid 0 !!!!

$P(Z | X) = 0.73$ (Apply Gaussian Dist)

Inference :

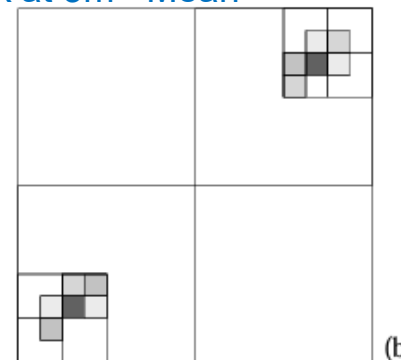
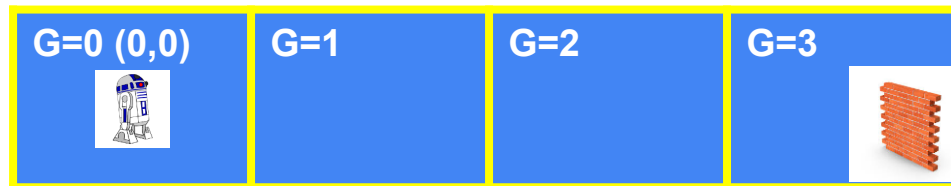
Most likely the robot is not in Grid 0 !!!!

Grid	Mean	P(Z X) 0.1	P(Z X) 0.5
0	3m	0.003	0.73
1	2m	~0	0.37
2	1m	~0	0.03

Coarse Grained Representation with Dynamic Grid Structures

- Topological Representation
- Inflate Motion Noise
- **Inflate Measurement Noise**
 - 1m Grid
 - At t=2, Robot is at 0.4m and senses landmark at 2.6m
 - Keep sensor noise = 0.1
 - Landmark at 3m= Mean

Add Noise to Motion Model Eg., 0.5



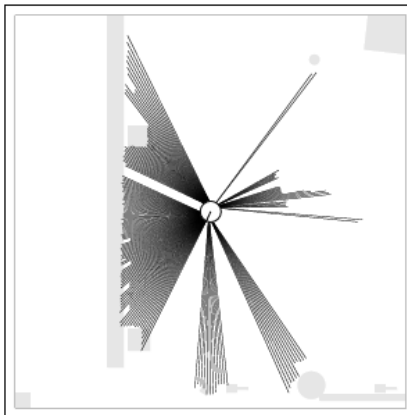
Grid Based Localization :



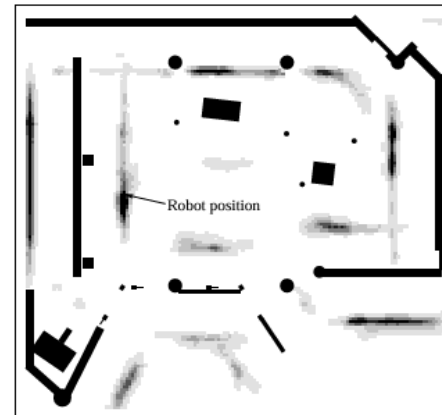
Illustration

Set up 1:
15 centimeter and an
angular resolution of 5
degrees

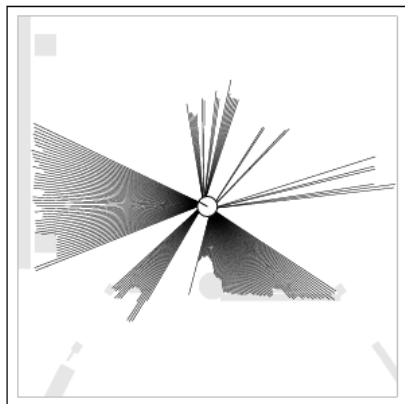
Two Laser Finders
Bean Sensor Model
Subsampling



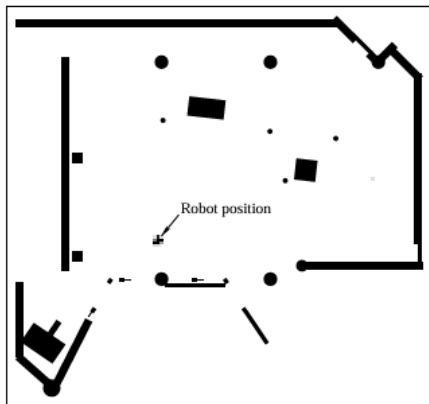
(a)



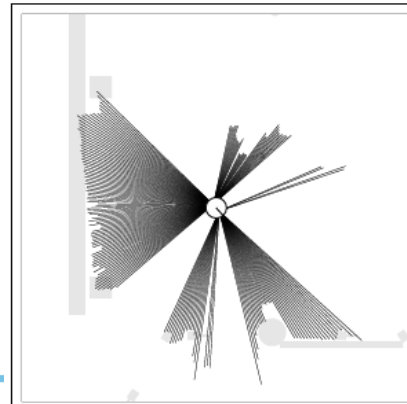
(b)



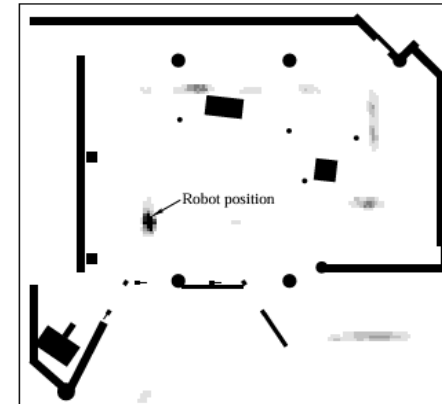
(c)



(d)



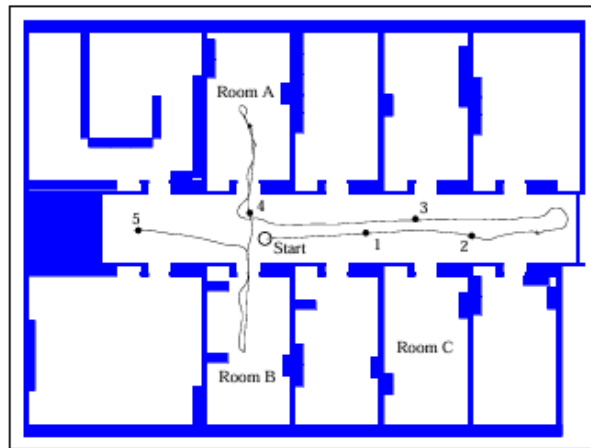
(e)



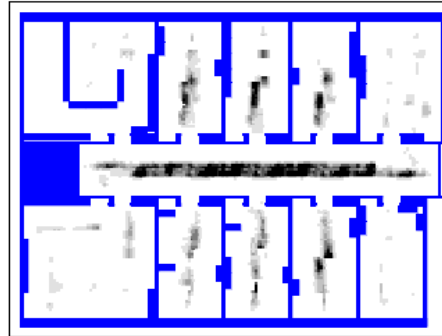
(f)

Grid Based Localization :

Illustration



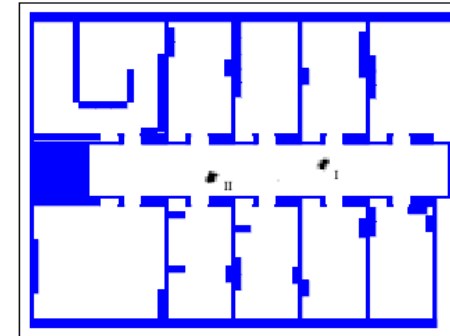
(a)



(b)



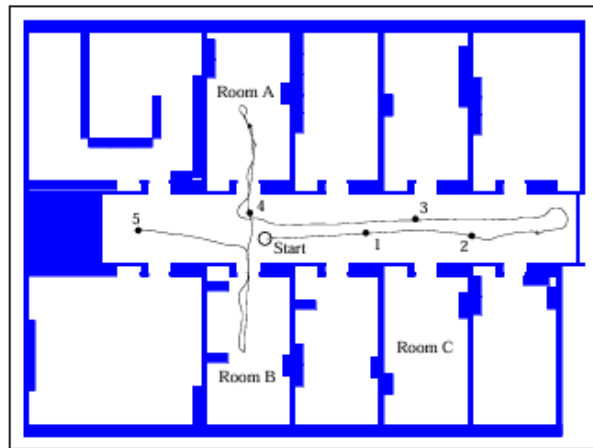
(c)



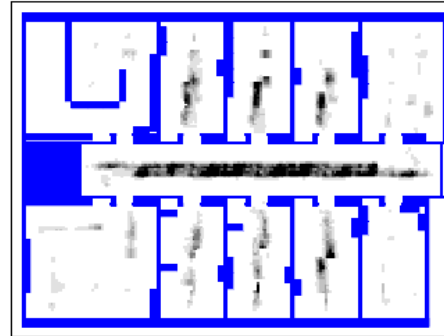
(d)

Grid Based Localization :

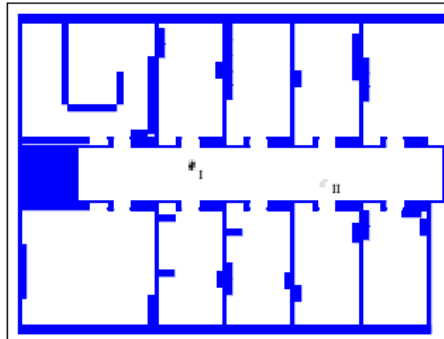
Illustration



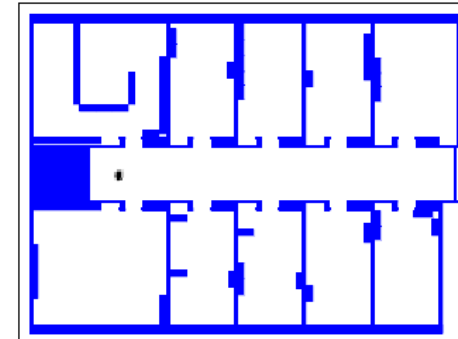
(a)



(b)



(e)



(f)

AI/ML in Grid-Based Localization, Specifically

Four very concrete places it pays off

01

Fusing many noisy sensors

LIDAR + camera + IMU + WiFi together: train a net mapping (cell, raw packet) $\rightarrow$ likelihood, plug into the histogram update.

USE CASE $\rightarrow$ multi-sensor mobile robot

02

Camera streams as measurements

A frame is too high-dim for an analytical likelihood. A CNN outputs a heatmap over cells — that heatmap IS $p(z_t|x_t)$.

USE CASE $\rightarrow$ visual localization, no LIDAR

03

Action recommendation (active)

Which action maximises info gain? Classical entropy calc is slow; RL learns a policy $\pi(u | \text{bel})$ directly.

USE CASE $\rightarrow$ kidnapped-robot recovery

04

Learning the motion noise

Instead of hand-picking 20% / 50% inflation, train a model to predict per-cell noise from odometry vs ground truth.

USE CASE $\rightarrow$ AMR on slippery surfaces

Required Reading

- T1 Chapter #4, #7, #8

T2 Chapter #6.1, #6.2

Next Class Plan

- ✓ Monte Carlo Based Localization